Homework 07

Author: Manushri Pendekanti (mp79)

In [1]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

%matplotlib inline
```

In [2]:
```
#Setup

import numpy as np
import pandas as pd

url = "data/saheart.csv"
df = pd.read_csv(url)

df = df.loc[:, ~df.columns.str.contains(r"^Unnamed")]

predictors = [
    "sbp", "tobacco", "ldl", "adiposity", "famhist",
    "typea", "obesity", "alcohol", "age"
]

# Code family history as requested
df["famhist"] = df["famhist"].map({"Absent": 1, "Present": 2})

X = df[predictors].to_numpy(dtype=float)
y = df["chd"].to_numpy(dtype=int)

classes = np.sort(np.unique(y))
n, d = X.shape
K = len(classes)
```

Question 1: LDA

Question 1 — Part a

(Your written answer.)

In [3]:
```
priors = np.array([(y == k).mean() for k in classes])


means = np.vstack([
    X[y == k].mean(axis=0)
    for k in classes
```

```
])


Sigma = np.zeros((d, d))

for j, k in enumerate(classes):
    Xk = X[y == k]
    centered = Xk - means[j]
    Sigma += centered.T @ centered

Sigma /= (n - K)

print("Priors:", priors)
print("Class means:\n", means)
print("Pooled covariance:\n", Sigma)

print("Dimensions:")
print("X:", X.shape)
print("priors:", priors.shape)
print("means:", means.shape)
print("Sigma:", Sigma.shape)
```

```
Priors: [0.65367965 0.34632035]
Class means:
 [[135.4602649    2.6347351    4.34423841  23.96910596   1.31788079
   52.36754967  25.73745033  15.93135762  38.85430464]
 [143.7375       5.524875     5.4879375   28.12025      1.6
   54.49375     26.6229375   19.14525     50.29375   ]]
Pooled covariance:
 [[ 4.05434779e+02  1.45851882e+01  4.58124834e+00  4.91643582e+01
   3.36988195e-01 -1.55877215e+01  1.89388430e+01  6.44003517e+01
   9.51338895e+01]
 [ 1.45851882e+01  1.92425564e+01  7.63202713e-01  7.53810931e+00
   1.58205010e-02 -2.05730592e+00  1.83345055e+00  2.05170451e+01
   2.27654113e+01]
 [ 4.58124834e+00  7.63202713e-01  4.00058129e+00  6.03269701e+00
   9.18502447e-02  3.44592335e-01  2.66005159e+00 -2.53290675e+00
   6.47892049e+00]
 [ 4.91643582e+01  7.53810931e+00  6.03269701e+00  5.67528931e+01
   4.32808321e-01 -5.30956633e+00  2.27078852e+01  1.61190253e+01
   6.05086715e+01]
 [ 3.36988195e-01  1.58205010e-02  9.18502447e-02  4.32808321e-01
   2.25833573e-01  8.11200691e-02  1.84027757e-01  7.68468845e-01
   9.97362511e-01]
 [-1.55877215e+01 -2.05730592e+00  3.44592335e-01 -5.30956633e+00
   8.11200691e-02  9.55656429e+01  2.64006728e+00  7.96003164e+00
  -2.02783349e+01]
 [ 1.89388430e+01  1.83345055e+00  2.66005159e+00  2.27078852e+01
   1.84027757e-01  2.64006728e+00  1.76154236e+01  4.68935626e+00
   1.56969995e+01]
 [ 6.44003517e+01  2.05170451e+01 -2.53290675e+00  1.61190253e+01
   7.68468845e-01  7.96003164e+00  4.68935626e+00  5.98276613e+02
   2.78859413e+01]
 [ 9.51338895e+01  2.27654113e+01  6.47892049e+00  6.05086715e+01
   9.97362511e-01 -2.02783349e+01  1.56969995e+01  2.78859413e+01
   1.84132137e+02]]
Dimensions:
X: (462, 9)
priors: (2,)
means: (2, 9)
Sigma: (9, 9)
```

Question 1 — Part b

(Your written answer.)

In [4]:

```python
from sklearn.metrics import confusion_matrix

scores = np.zeros((n, K))

for j in range(K):
    mu = means[j]

    w = np.linalg.solve(Sigma, mu)

    scores[:, j] = (
```

```
        X @ w
        - 0.5 * (mu @ w)
        + np.log(priors[j])
    )


pred_manual = classes[np.argmax(scores, axis=1)]

cm_manual = confusion_matrix(y, pred_manual, labels=classes)

print("Manual confusion matrix:")
print(cm_manual)
```

```
Manual confusion matrix:
[[258  44]
 [ 73  87]]
```

Question 1 — Part c

(Your written answer.)

In [5]:
```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

lda = LinearDiscriminantAnalysis(
    solver="svd",
    priors=priors
)

lda.fit(X, y)
pred_sklearn = lda.predict(X)

cm_sklearn = confusion_matrix(y, pred_sklearn, labels=classes)

print("sklearn confusion matrix:")
print(cm_sklearn)

print("Do the predictions match?",
      np.array_equal(pred_manual, pred_sklearn))
```

```
sklearn confusion matrix:
[[258  44]
 [ 73  87]]
Do the predictions match? True
```

In [6]:
```
#Setup

import numpy as np
import pandas as pd
from sklearn.discriminant_analysis import QuadraticDiscriminantAnalysis
from sklearn.metrics import confusion_matrix

train_url = "data/zip-train.csv.gz"
test_url  = "data/zip-test.csv.gz"
```

```
train = pd.read_csv(train_url, header=None, compression="gzip").to_numpy()
test = pd.read_csv(test_url, header=None, compression="gzip").to_numpy()

y_train = train[:, 0].astype(int)
X_train = train[:, 1:].astype(float)

y_test = test[:, 0].astype(int)
X_test = test[:, 1:].astype(float)

# Rank pixels by their variance across the training observations.
pixel_variances = np.var(X_train, axis=0, ddof=1)
ranked_pixels = np.argsort(pixel_variances)[::-1]

print("Training shape:", X_train.shape)
print("Test shape:", X_test.shape)
```

```
Training shape: (7291, 256)
Test shape: (2007, 256)
```

Question 2: QDA and marginal screening

Question 2 — Part a

(Your written answer.)

In [7]:
```
pixels_40 = ranked_pixels[:40]

qda_40 = QuadraticDiscriminantAnalysis(tol=1e-8)
qda_40.fit(X_train[:, pixels_40], y_train)

pred_40 = qda_40.predict(X_test[:, pixels_40])

cm_40 = confusion_matrix(y_test, pred_40, labels=np.arange(10))
error_40 = np.mean(pred_40 != y_test)

print("40-pixel confusion matrix (rows = observed, columns = predicted):")
print(cm_40)
print("40-pixel test misclassification rate:", error_40)
```

```
40-pixel confusion matrix (rows = observed, columns = predicted):
[[319   0  24   6   2   2   1   0   5   0]
 [  0 238   5   0  11   0   7   0   3   0]
 [  8   0 168   8   8   3   0   0   3   0]
 [  5   0   4 128   0  14   0   0  15   0]
 [  0   2  13   1 150   1   0   1   7  25]
 [ 12   0   3   5   1 131   0   0   6   2]
 [  0   0   8   0   1   8 148   0   5   0]
 [  1   0   6   1   6   0   0 124   2   7]
 [  1   0  10  12   3   2   0   0 137   1]
 [  0   0   1   2   7   1   1   4   9 152]]
40-pixel test misclassification rate: 0.1554559043348281
```

Question 2 — Part b

(Your written answer.)

In [8]:
```
#the 40 pixel verision can work because QDA seperate covariance for each cla
#pixels make covariance estimates less likely to be singular
```

Question 2 — Part c

(Your written answer.)

In [9]:
```
pixels_10 = ranked_pixels[:10]

qda_10 = QuadraticDiscriminantAnalysis(tol=1e-8)
qda_10.fit(X_train[:, pixels_10], y_train)

pred_10 = qda_10.predict(X_test[:, pixels_10])

cm_10 = confusion_matrix(y_test, pred_10, labels=np.arange(10))
error_10 = np.mean(pred_10 != y_test)

print("10-pixel confusion matrix (rows = observed, columns = predicted):")
print(cm_10)
print("10-pixel test misclassification rate:", error_10)

print("40-pixel error rate:", error_40)
print("10-pixel error rate:", error_10)
```

```
10-pixel confusion matrix (rows = observed, columns = predicted):
[[298   0  11   1   1  16  28   4   0   0]
 [  0 241  10   0   3   1   1   0   2   6]
 [ 19   2  95  14  14   3  22  21   5   3]
 [  5   0   6 103   3  42   0   0   7   0]
 [  1   5  14   0  95   2   4  15  17  47]
 [ 15   0   6  25   4  96   4   2   8   0]
 [ 41   0  15   4   0  18  78   7   5   2]
 [  4   1   6   0   6   0   1 122   0   7]
 [  1   2  11  36  12  11   4   1  74  14]
 [  0   2   9   2  20   0   0  16  10 118]]
10-pixel test misclassification rate: 0.3423019431988042
40-pixel error rate: 0.1554559043348281
10-pixel error rate: 0.3423019431988042
```

Question 3: Compare LDA and QDA

In [10]:
```
#setup

import numpy as np
import matplotlib.pyplot as plt
```

```
from sklearn.discriminant_analysis import (
    LinearDiscriminantAnalysis,
    QuadraticDiscriminantAnalysis
)

seed = 412
rng = np.random.default_rng(seed)

rhos = [0, 0.2, 0.4, 0.6, 0.8, 0.9]

def generate_data(rho, n_per_class, rng):
    # Covariance for the first two variables in each class
    cov_class_1 = np.array([
        [1,  rho],
        [rho, 1]
    ])

    cov_class_2 = np.array([
        [1,  -rho],
        [-rho, 1]
    ])

    # Add eight independent standard normal variables
    cov1 = np.eye(10)
    cov1[:2, :2] = cov_class_1

    cov2 = np.eye(10)
    cov2[:2, :2] = cov_class_2

    mean1 = np.array([-1, 0, 0, 0, 0, 0, 0, 0, 0, 0])
    mean2 = np.array([ 1, 0, 0, 0, 0, 0, 0, 0, 0, 0])

    X1 = rng.multivariate_normal(mean1, cov1, size=n_per_class)
    X2 = rng.multivariate_normal(mean2, cov2, size=n_per_class)

    X = np.vstack((X1, X2))
    y = np.array([1] * n_per_class + [2] * n_per_class)

    # Shuffle the observations
    order = rng.permutation(len(y))
    return X[order], y[order]
```

Question 3 — Part a

(Your written answer.)

In [11]:

```
one_dataset_errors = []

for rho in rhos:
    X_train, y_train = generate_data(rho, 100, rng)
    X_test, y_test = generate_data(rho, 1000, rng)

    lda = LinearDiscriminantAnalysis(
```

```
        solver="lsqr",
        priors=[0.5, 0.5]
    )
    qda = QuadraticDiscriminantAnalysis(
        priors=[0.5, 0.5]
    )

    lda.fit(X_train, y_train)
    qda.fit(X_train, y_train)

    lda_error = np.mean(lda.predict(X_test) != y_test)
    qda_error = np.mean(qda.predict(X_test) != y_test)

    one_dataset_errors.append((lda_error, qda_error))
    print(
        f"rho={rho:.1f}: "
        f"LDA error={lda_error:.4f}, "
        f"QDA error={qda_error:.4f}"
    )
```

```
rho=0.0: LDA error=0.1660, QDA error=0.1985
rho=0.2: LDA error=0.1695, QDA error=0.2240
rho=0.4: LDA error=0.1555, QDA error=0.1750
rho=0.6: LDA error=0.1540, QDA error=0.1740
rho=0.8: LDA error=0.1700, QDA error=0.1310
rho=0.9: LDA error=0.1735, QDA error=0.1080
```

Question 3 — Part b

(Your written answer.)

In [12]:

```
average_errors = []

for rho in rhos:
    repetition_errors = []

    for repetition in range(50):
        X_train, y_train = generate_data(rho, 100, rng)
        X_test, y_test = generate_data(rho, 1000, rng)

        lda = LinearDiscriminantAnalysis(
            solver="lsqr",
            priors=[0.5, 0.5]
        )
        qda = QuadraticDiscriminantAnalysis(
            priors=[0.5, 0.5]
        )

        lda.fit(X_train, y_train)
        qda.fit(X_train, y_train)

        lda_error = np.mean(lda.predict(X_test) != y_test)
        qda_error = np.mean(qda.predict(X_test) != y_test)
```

```
        repetition_errors.append([lda_error, qda_error])

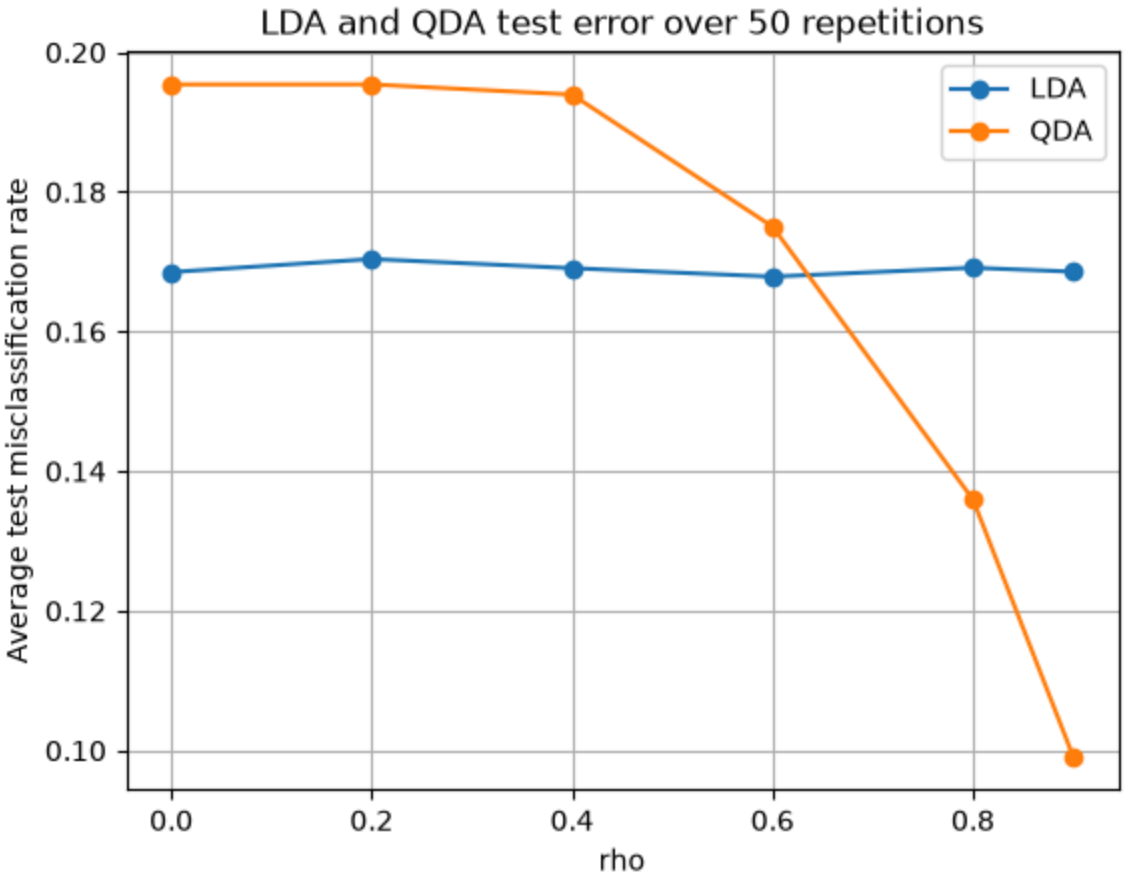
    average_errors.append(np.mean(repetition_errors, axis=0))

average_errors = np.array(average_errors)

for i, rho in enumerate(rhos):
    print(
        f"rho={rho:.1f}: "
        f"average LDA error={average_errors[i, 0]:.4f}, "
        f"average QDA error={average_errors[i, 1]:.4f}"
    )

plt.plot(rhos, average_errors[:, 0], marker="o", label="LDA")
plt.plot(rhos, average_errors[:, 1], marker="o", label="QDA")
plt.xlabel("rho")
plt.ylabel("Average test misclassification rate")
plt.title("LDA and QDA test error over 50 repetitions")
plt.legend()
plt.grid(True)
plt.show()
```

```
rho=0.0: average LDA error=0.1685, average QDA error=0.1954
rho=0.2: average LDA error=0.1704, average QDA error=0.1954
rho=0.4: average LDA error=0.1691, average QDA error=0.1939
rho=0.6: average LDA error=0.1679, average QDA error=0.1750
rho=0.8: average LDA error=0.1691, average QDA error=0.1360
rho=0.9: average LDA error=0.1686, average QDA error=0.0991
```



Question 3 — Part c

(Your written answer.)

In [13]:
```
#when p = 0 the clases have same covariance matrix
#lda estimates fewer covariance params
#qda estimates for convariance matrix for each class
#which adds flexibility but also noise
#as p become larger covariance matrix more different
# which QDA can represent and not LDA
```

Question 4: Create and use an LDA/QDA analysis skill

Part a

Created `lda-qda-analysis/SKILL.md` (3,705 bytes). Skill requires explicit invocation and compares LDA, QDA, PCA extensions, and probability blends.

Part b

Invoked the skill on SAheart with seed 43207. Code and results below.

In [14]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, StratifiedKFold
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis, Quadra
from sklearn.pipeline import Pipeline
from sklearn.metrics import confusion_matrix

SEED = 43207
np.random.seed(SEED)

heart = pd.read_csv("data/saheart.csv")
heart["famhist"] = heart["famhist"].map({"Absent": 1, "Present": 2})
X = heart.drop(columns="chd")
y = heart["chd"].to_numpy()

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=SEED, stratify=y
)

cv = StratifiedKFold(n_splits=10, shuffle=True, random_state=SEED)

def misclass_rate(y_true, y_pred):
    return np.mean(y_true != y_pred)
```

```
def cv_misclass(clf, X_tr, y_tr):
    scores = []
    for tr_idx, va_idx in cv.split(X_tr, y_tr):
        X_fold_tr, X_fold_va = X_tr.iloc[tr_idx], X_tr.iloc[va_idx]
        y_fold_tr, y_fold_va = y_tr[tr_idx], y_tr[va_idx]
        clf.fit(X_fold_tr, y_fold_tr)
        scores.append(misclass_rate(y_fold_va, clf.predict(X_fold_va)))
    return np.array(scores)

class BlendLDAQDA:
    def __init__(self, weight=0.5):
        self.weight = weight
        self.scaler = StandardScaler()
        self.lda = LinearDiscriminantAnalysis()
        self.qda = QuadraticDiscriminantAnalysis()
    def fit(self, X, y):
        Xs = self.scaler.fit_transform(X)
        self.lda.fit(Xs, y)
        self.qda.fit(Xs, y)
        self.classes_ = self.lda.classes_
        return self
    def predict_proba(self, X):
        Xs = self.scaler.transform(X)
        p = self.weight * self.lda.predict_proba(Xs) + (1 - self.weight) * s
        return p
    def predict(self, X):
        return self.classes_[np.argmax(self.predict_proba(X), axis=1)]

candidates = {
    "LDA": Pipeline([("scale", StandardScaler()), ("lda", LinearDiscriminant
    "QDA": Pipeline([("scale", StandardScaler()), ("qda", QuadraticDiscrimir
    "PCA(5)+LDA": Pipeline([("scale", StandardScaler()), ("pca", PCA(5)), ("
    "LDA–QDA blend (0.5)": BlendLDAQDA(0.5),
}

results = {name: cv_misclass(clf, X_train, y_train) for name, clf in candida
cv_table = pd.DataFrame({
    "Model": list(results.keys()),
    "Mean CV error": [v.mean() for v in results.values()],
    "SD CV error": [v.std(ddof=1) for v in results.values()],
})
display(cv_table)

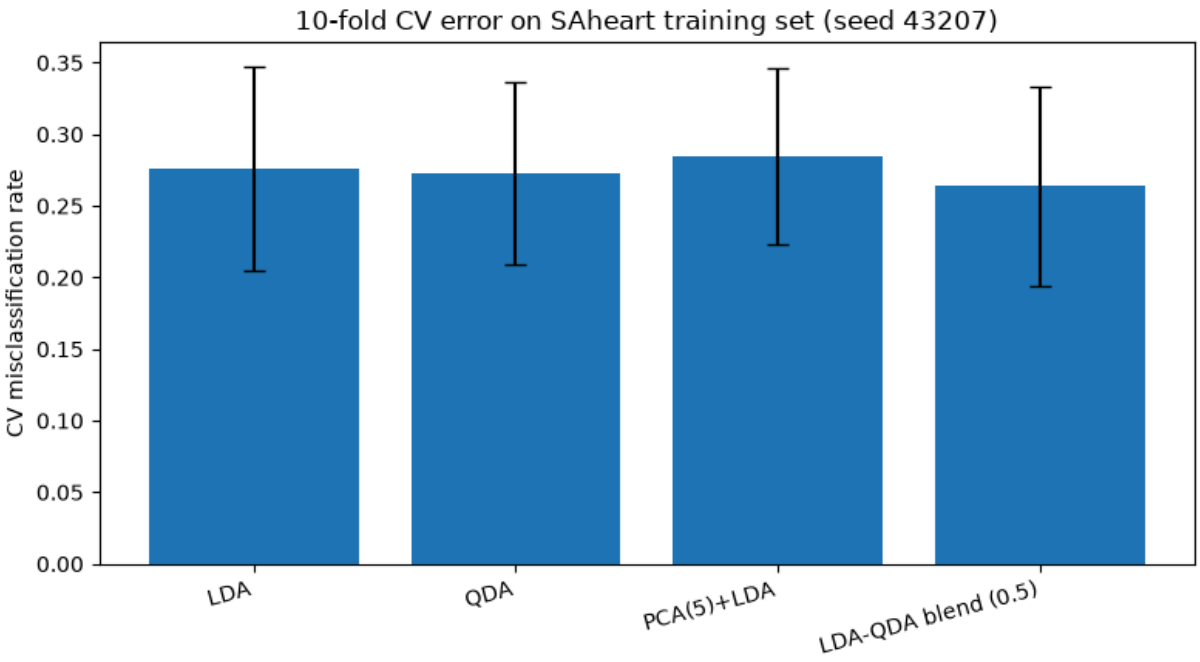
best_name = cv_table.loc[cv_table["Mean CV error"].idxmin(), "Model"]
best_clf = candidates[best_name]
best_clf.fit(X_train, y_train)
y_pred_test = best_clf.predict(X_test)
print("Selected model:", best_name)
print("Test misclassification rate:", misclass_rate(y_test, y_pred_test))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred_test))

fig, ax = plt.subplots(figsize=(8, 4.5))
means = cv_table["Mean CV error"].to_numpy()
sds = cv_table["SD CV error"].to_numpy()
x = np.arange(len(cv_table))
```

```
ax.bar(x, means, yerr=sds, capsize=5)
ax.set_xticks(x)
ax.set_xticklabels(cv_table["Model"], rotation=15, ha="right")
ax.set_ylabel("CV misclassification rate")
ax.set_title("10-fold CV error on SAheart training set (seed 43207)")
fig.tight_layout()
fig.savefig("q4-cv-errors.png", dpi=150)
plt.show()
```

	Model	Mean CV error	SD CV error
0	LDA	0.275663	0.071152
1	QDA	0.272917	0.063687
2	PCA(5)+LDA	0.284848	0.061437
3	LDA-QDA blend (0.5)	0.263636	0.069397

```
Selected model: LDA-QDA blend (0.5)
Test misclassification rate: 0.26618705035971224
Confusion matrix:
 [[77 14]
 [23 25]]
```



LDA/QDA analysis skill

```
---
name: stat432-lda-qda-analysis
description: Runs an LDA/QDA classification workflow with cross-
validation and a one-page report. Use only when the user explicitly
invokes this skill and supplies the dataset, response, and
predictors.
disable-model-invocation: true
---
```

STAT 432 LDA/QDA Analysis

Intent

Guide a reproducible discriminant-analysis study using only LDA, QDA, or combinations of their class-probability outputs. Produce a concise one-page report with one figure, one table, and numerical fit results.

When to use

Apply this skill only when the student explicitly names `stat432-lda-qda-analysis` or asks you to use this LDA/QDA analysis skill. The student must supply:

- dataset (file path or loaded table)
- response column or vector (binary or multiclass labels)
- predictor columns or matrix (numeric)
- random seed (optional; ask if missing)

Do not assume variable names, class labels, or expected performance.

Analysis workflow

1. **Split first.** Hold out a test set (default 30%) with stratification when possible. Fix the split with the stated seed. Never use the test set for model or preprocessing choices.

2. **Define candidates.** Compare at minimum:
   - ordinary LDA (shared covariance)
   - ordinary QDA (class-specific covariances)
   Optional extensions allowed by this skill:
   - PCA embedding + LDA or QDA (number of components chosen by CV)
   - blended classifier: average or weighted average of LDA and QDA predicted class probabilities, then classify by largest blended probability

3. **Cross-validate on training only.** Use one fixed fold scheme (default: 10-fold stratified). Within each fold:
   - fit preprocessing (standardization, PCA, etc.) on training folds only
   - fit each candidate on preprocessed training folds
   - record validation misclassification rate
   Summarize mean CV error (and SD if helpful) per candidate. Select the best candidate by lowest mean CV misclassification rate.

4. **Final fit and test evaluation.** Refit preprocessing and the selected candidate on all training data. Evaluate once on the held-out test set. Report test misclassification rate and a confusion matrix.

```
5. **Keep classifiers in scope.** Do not use logistic regression,
trees, SVM, or neural networks. Only LDA, QDA, PCA variants of
these, or probability blends of LDA and QDA.

## Required one-page report

Fit on one printed page (~400–500 words plus one figure and one
table):

| Section | Content |
|---------|---------|
| Setup | Sample size, number of predictors, train/test split,
seed, response definition |
| Candidates | Brief description of each model compared |
| Cross-validation | Fold scheme, metric (misclassification rate),
CV results table |
| Selection | Chosen model and why (CV performance) |
| Test results | Test misclassification rate, key numerical outputs
(priors, dimensions, tuning choices) |
| Figure | One plot (e.g., CV error by candidate, or 2D score/PC
scatter with classes) |
| Findings | Main conclusion and one limitation |

## Figure and table rules

- **Table:** CV misclassification rates for all candidates (mean ±
SD across folds).
- **Figure:** One clear plot supporting model comparison or class
separation; label axes; readable font size.
- **Numbers:** Report seed, test error, selected model name, and at
least one fitted statistic (e.g., estimated priors or number of PCA
components).

## Rules

- Do not peek at the test set until the final evaluation step.
- Do not embed dataset-specific names or expected results in this
skill.
- If the report exceeds one page, shorten prose before removing the
required figure or table.
- State which Python/R functions were used for LDA and QDA.
```

One-page analysis report

Setup

- **Data:** SAheart, n = 462 observations, 9 numeric predictors (`famhist` coded 1 = Absent, 2 = Present)

- **Response:** `chd` (0 = no CHD, 1 = CHD)
- **Split:** 70% train / 30% test, stratified, **seed 43207** → 323 train, 139 test
- **Metric:** misclassification rate

Candidate models

1. **LDA** — shared covariance (`LinearDiscriminantAnalysis` after standardization)
2. **QDA** — class-specific covariances (`QuadraticDiscriminantAnalysis` after standardization)
3. **PCA(5) + LDA** — 5 principal components then LDA
4. **LDA–QDA blend** — 50/50 average of class probabilities from LDA and QDA

Cross-validation (training only)

10-fold stratified CV on 323 training observations. Preprocessing refit inside each fold.

Model	Mean CV error	SD
LDA	0.276	0.071
QDA	0.273	0.064
PCA(5)+LDA	0.285	0.061
LDA–QDA blend (0.5)	**0.264**	**0.069**

Selection

LDA–QDA blend (weight 0.5) had the lowest mean CV misclassification rate (0.264).

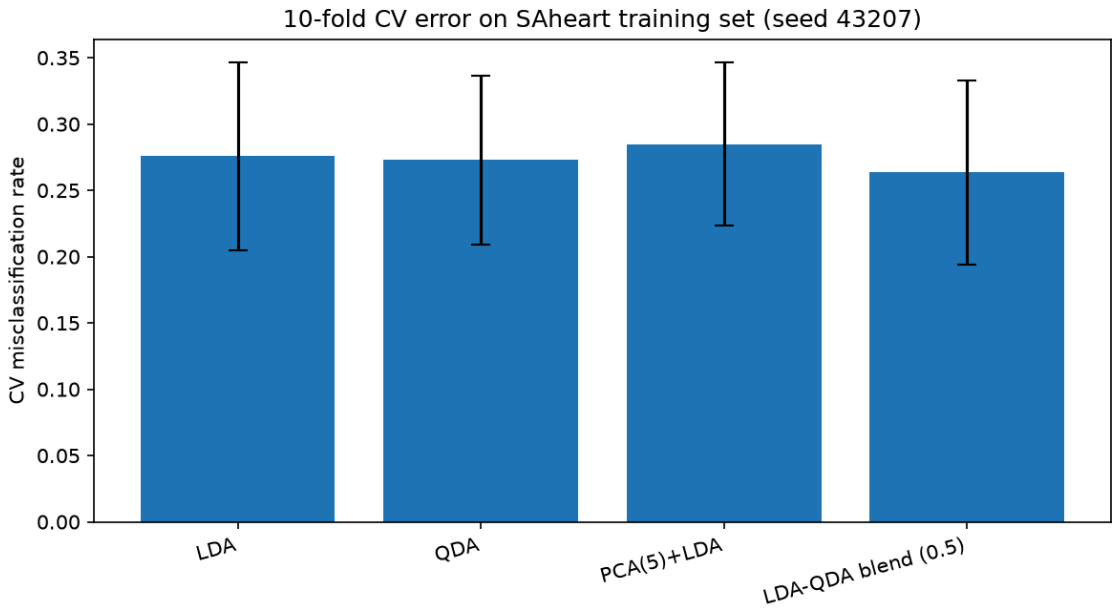
Test results

- **Test misclassification rate:** 0.266 (37/139 wrong)
- **Confusion matrix** (rows = true, cols = predicted):

	Pred 0	Pred 1
True 0	77	14
True 1	23	25

- **LDA priors on full training set:** $\hat{\pi}_0 = 0.653$, $\hat{\pi}_1 = 0.347$

Figure



Findings

Blending LDA and QDA gave a small CV improvement over either method alone. On the held-out test set, error was about 26.6%, with more false negatives (23) than false positives (14) for CHD.

Limitation: Only one blend weight (0.5) and one PCA dimension (5) were tried.

Part C — Skill submission notes

- **Skill file size:** 3,705 bytes (under 5 KB limit)
- **Explicit invocation:** `disable-model-invocation: true` in frontmatter; skill installed at `~/.cursor/skills/stat432-lda-qda-analysis/`
- **Prompt used:** "Use stat432-lda-qda-analysis to analyze this dataset. Response: chd (0/1). Predictors: all other columns after coding famhist as 1/2. Random seed: 43207. Hold out 30% for test. Compare LDA, QDA, and at least one optional extension (PCA or blend). Select the best model by 10-fold CV misclassification rate on the training set only. Report the one-page analysis with figure and table."